

Employee Management System using JSON Server and Postman

Objective:

To configure the Fake Rest API using JSON Server and run the dbs.json file on port number 3000 to manage Employee Details as per the requirement guidelines.

Software Requirements:

- Node.js
- npm
- JSON Server
- Postman

Step 1: Create the dbs.json File

Created a dbs.json file containing employee records with attributes such as Employee ID, Name, Department, Designation, Salary, and Email. This file acts as a temporary database for the JSON Server.

```
{
  "employees": [
    {
      "eid": 101,
      "ename": "Varshith",
      "email": "varshith@abc.com",
      "mobile": 12345,
      "id": "1G11b4IkDiY"
    },
    {
      "eid": 102,
      "ename": "Rajesh",
      "email": "rajesh@abc.com",
      "mobile": 14567,
      "id": "YexZrPRVixk"
    },
    {
      "eid": 103,
      "ename": "Charith",
      "email": "charith@abc.com",
      "mobile": 987656,
      "id": "_Rj3gTv9qlI"
    }
  ]
}
```

Step 2: Start JSON Server

Opened the terminal in the project folder and executed the command to start the JSON Server on port 3000.

Screenshot 2: JSON Server Running

```
Microsoft Windows [Version 10.0.26200.8655]
(c) Microsoft Corporation. All rights reserved.

D:\StudentRestAPI>npx json-server dbs.json --port 3000
npm notice run npx
npm notice run json-server dbs.json --port 3000
JSON Server started on PORT :3000
Press CTRL-C to stop
Watching dbs.json...

( - ^ _ ^ - )

Index:
http://localhost:3000/

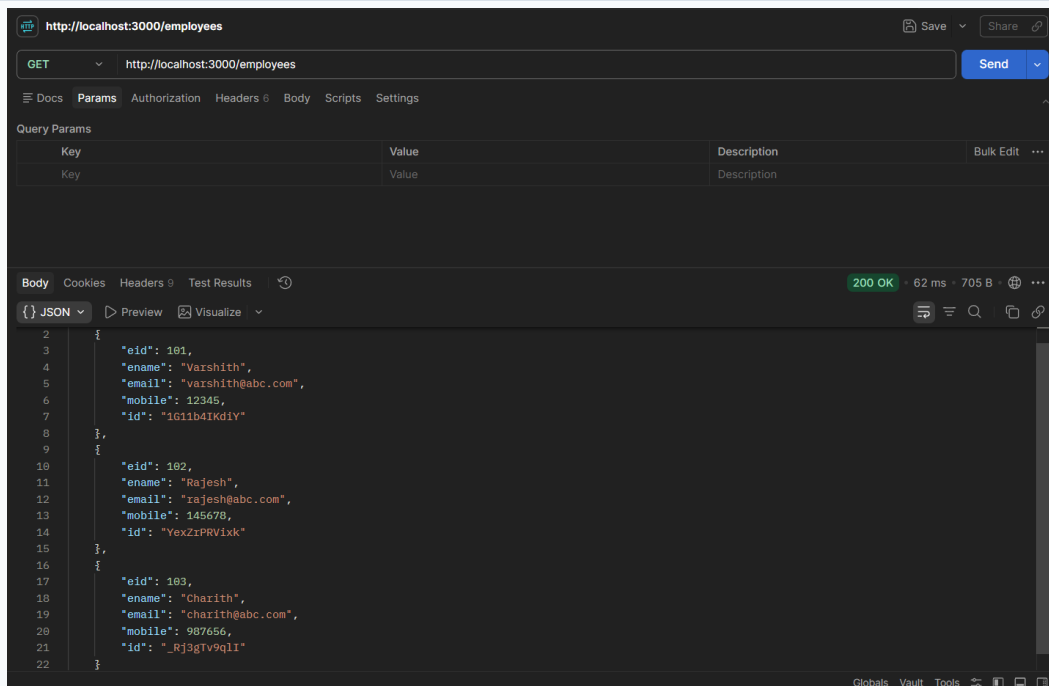
Static files:
Serving ./public directory if it exists

Endpoints:
http://localhost:3000/employees
|
```

Step 3: Test GET API (Retrieve All Employees)

Opened Postman and sent a GET request to retrieve all employee records stored in the JSON file.

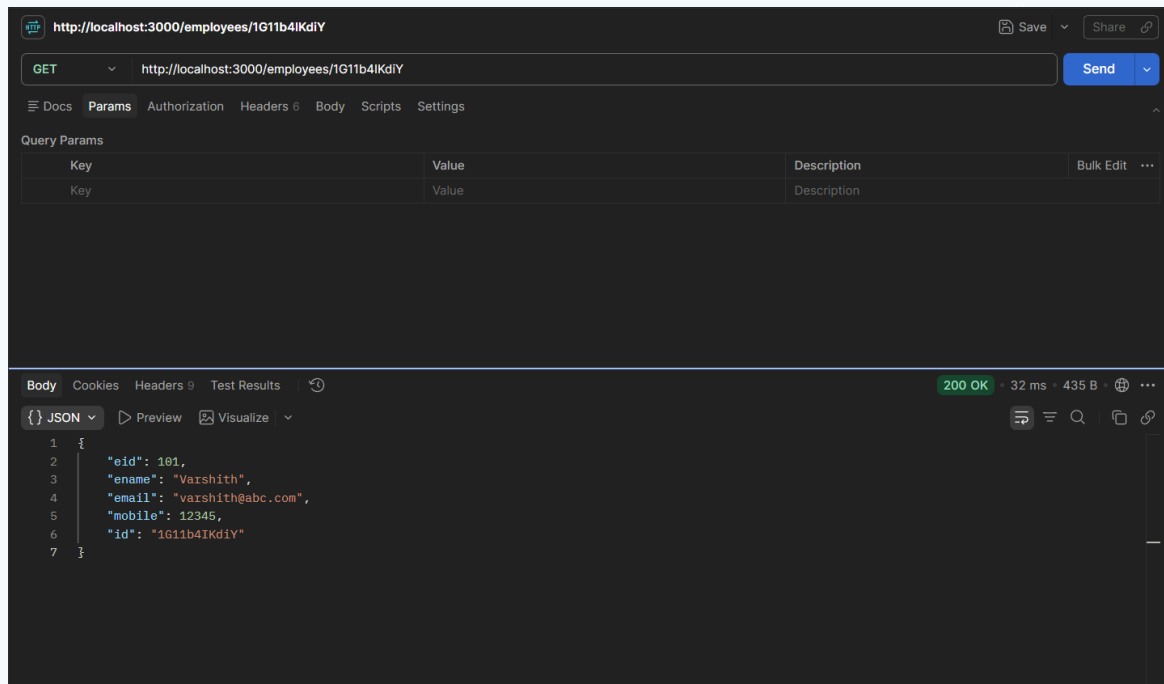
Screenshot 3: GET All Employees



Step 4: Test GET API (Retrieve Employee by ID)

Sent a GET request with a specific employee ID to retrieve the details of a single employee.

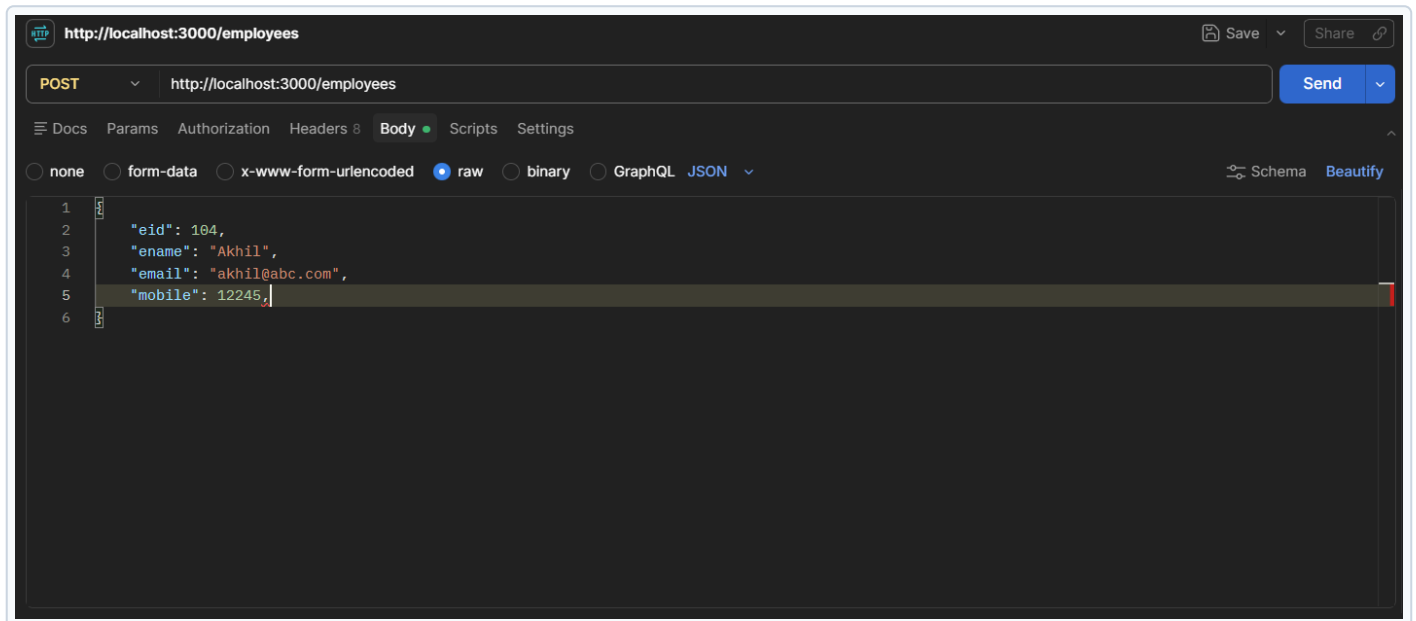
Screenshot 4: GET Employee by ID



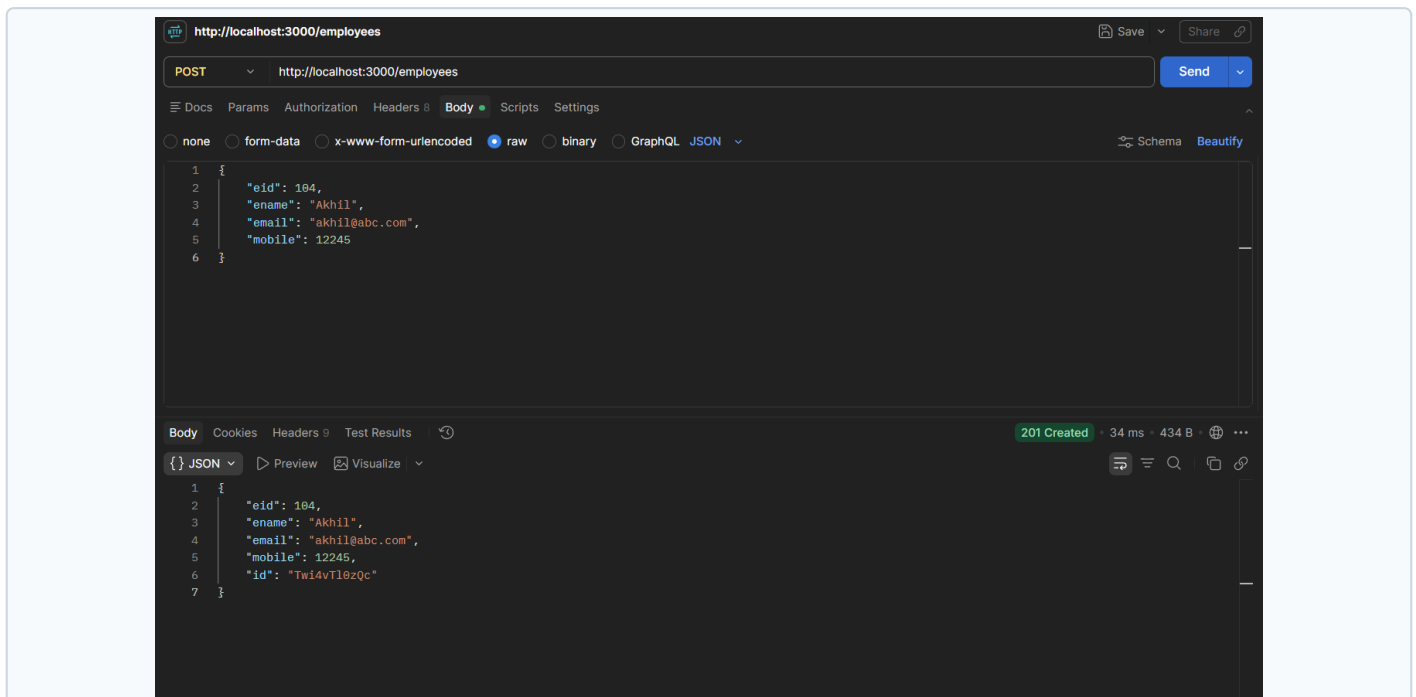
Step 5: Test POST API (Add New Employee)

Changed the request method to POST and sent the new record details inside the JSON body.

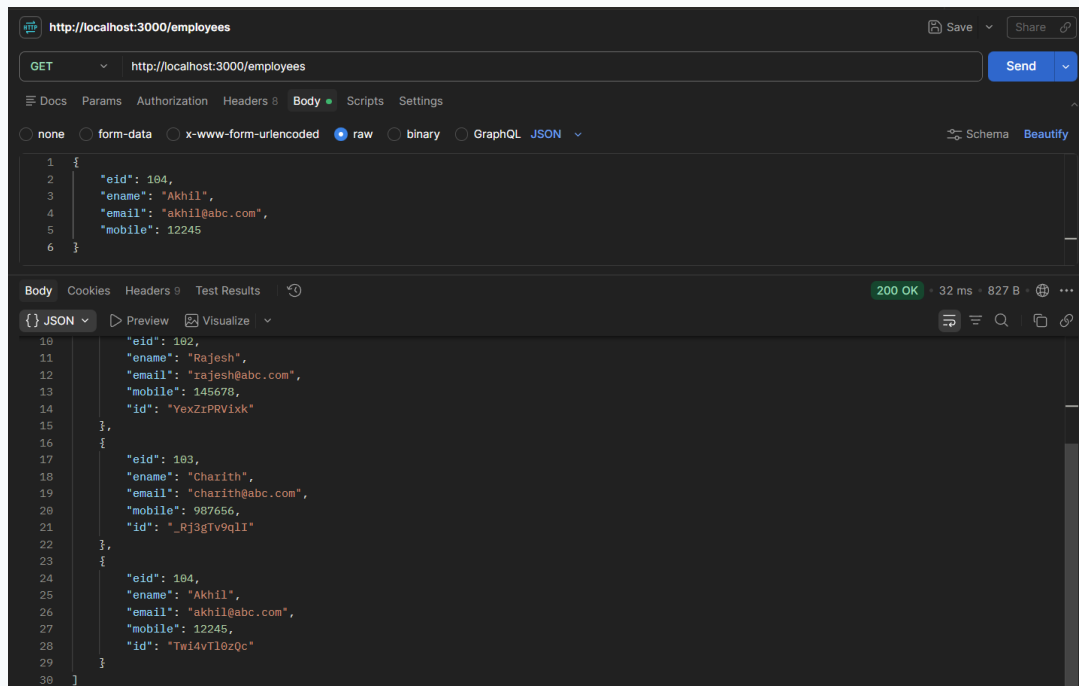
Screenshot 5 (a): POST Request Body Configuration



Screenshot 5 (b): POST Response (201 Created)



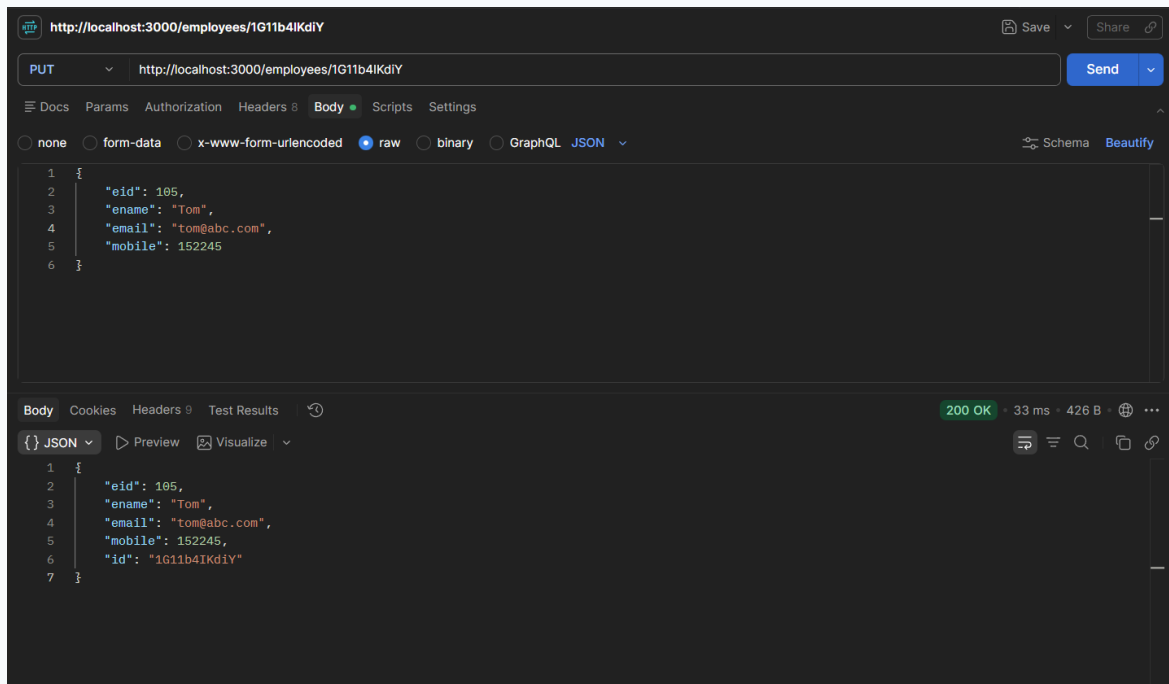
Screenshot 5 (c): Verification GET Request



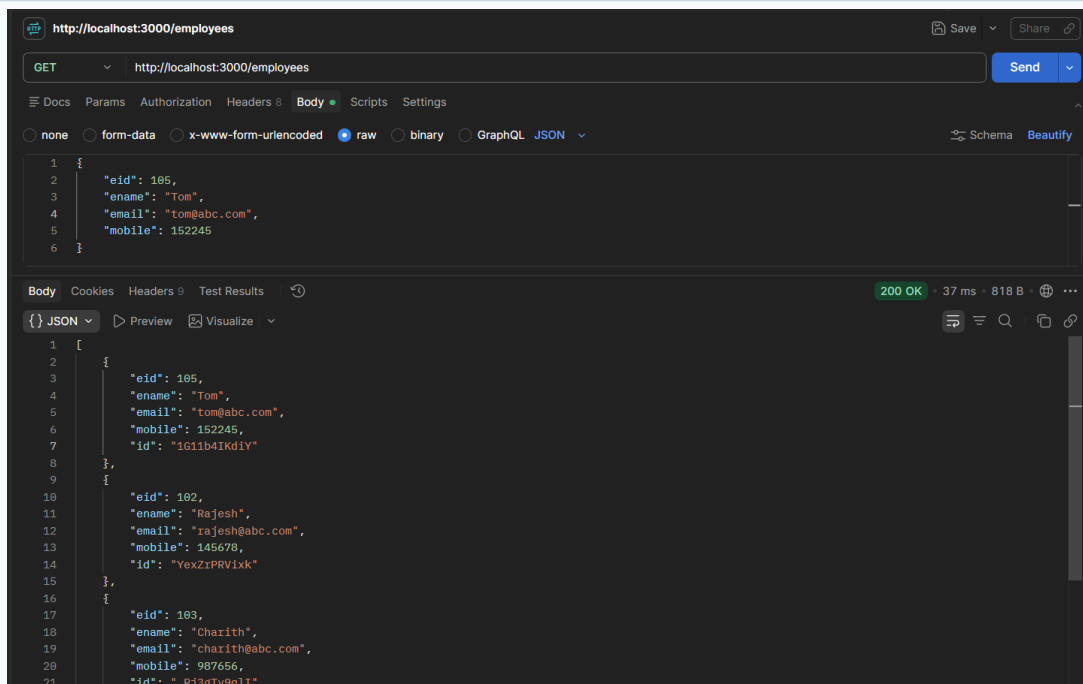
Step 6: Test PUT API (Update Employee Details)

Used the PUT request to completely update an existing record by providing the object structure.

Screenshot 6 (a): PUT Request Response



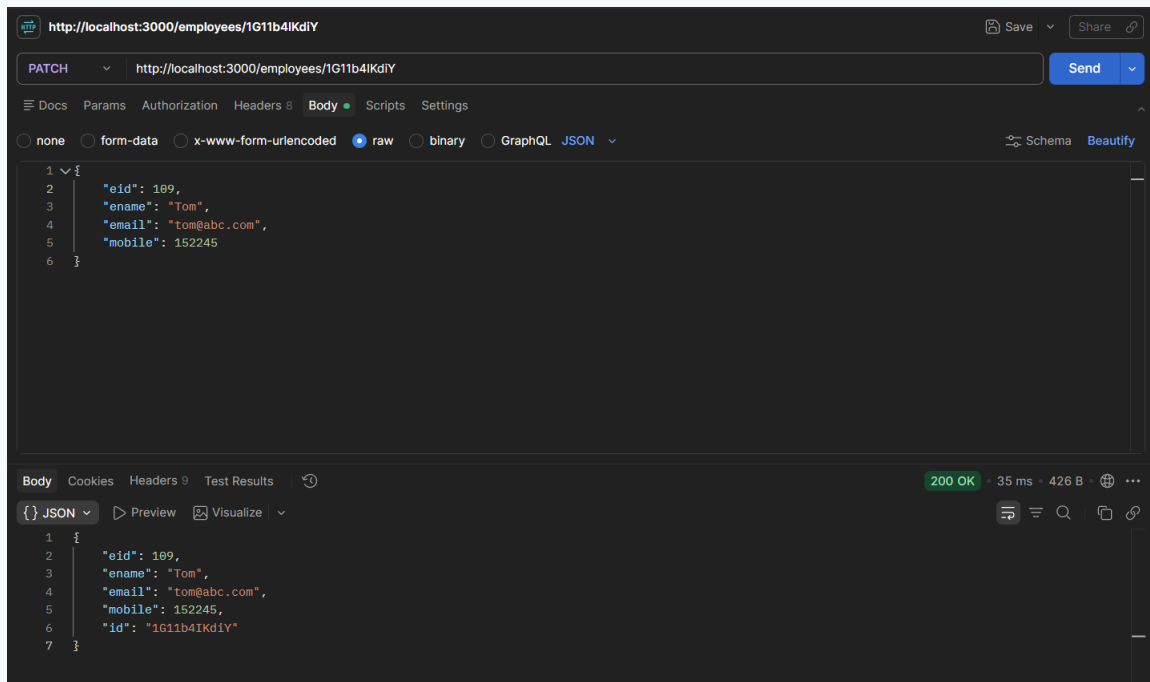
Screenshot 6 (b): GET All List Verification



Step 7: Test PATCH API (Update Specific Field)

Used the PATCH request to modify specific fields within a given employee record.

Screenshot 7: PATCH Response



Step 8: Test DELETE API (Delete Employee)

Sent a DELETE request targeting the employee's unique ID to remove it from the system database.

Screenshot 8: DELETE Response

